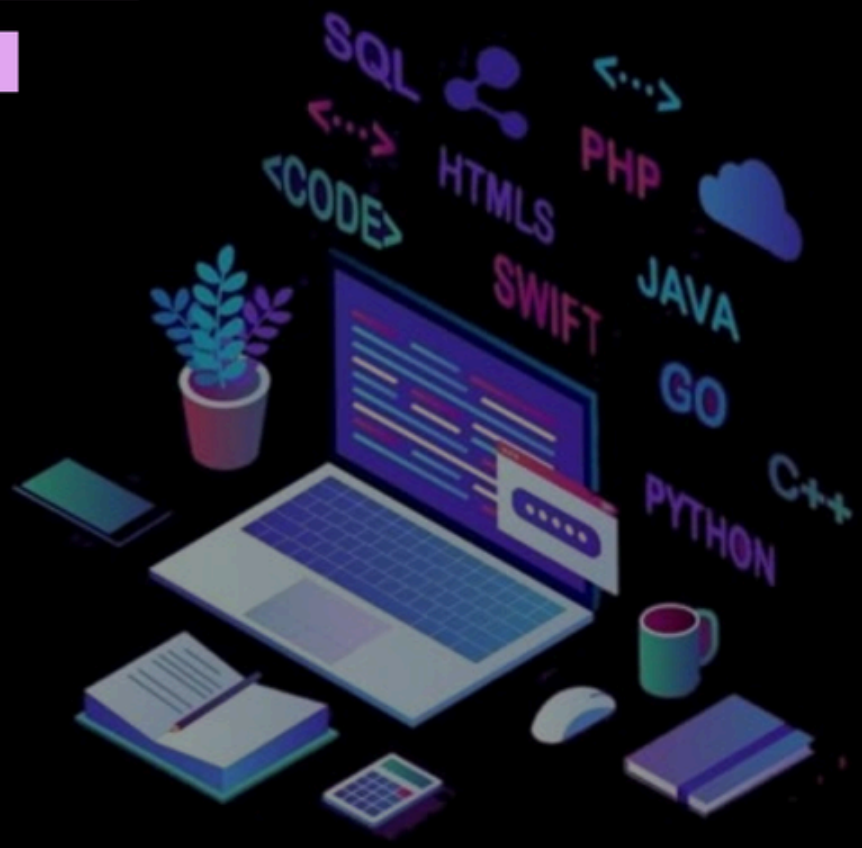
PROGRAMMERS CLUB

-DEPT. OF IT



The Programmers Club of the Information Technology Department is established to enhance the coding and problem-solving skills of students. It provides a platform to learn, collaborate, and innovate through coding quests, study materials, and practical solutions. The club aims to nurture technical excellence and prepare students for real-world challenges in the field of programming.

Faculty Co-ordinator :

Mrs. Pooja G. Dhawane

Student Co-ordinators:

Somesh Sharma

Om Kuthe

Nishant Lilhare

Tanmay Mankar

📘 Week 8 — Queue Applications

08

🧠 Quick Recap

Before learning applications, revise the core queue concepts:

🔹 FIFO Principle

Queue follows:

FIFO → First In, First Out

The **first element inserted** will be the **first element removed**.

🔹 Basic Operations

Operation	Description
Enqueue(x)	Insert element at the rear
Dequeue()	Remove element from the front
Front / Peek	View the first element
isEmpty()	Check if queue is empty
isFull()	Check if queue is full (array queue)

🔹 Front and Rear Movement

Example after inserting elements:

```
enqueue(10)
enqueue(20)
enqueue(30)
```

```
[10][20][30]
  ^        ^
front     rear
```

After one dequeue:

```
[10][20][30]
      ^    ^
   front  rear
```

🔹 When Should We Use a Queue?

Use a **Queue** when:

✔ Order of processing matters
✔ Elements must be processed in the same order they arrive
✔ Level-by-level processing is required
✔ Sequential checking is needed
✔ Sliding window problems appear

Common Situations

- Task scheduling
- Breadth First Search (BFS)
- Printer job queue
- Network packet processing
- Real-time stream processing

🔹 Applications Covered This Week

✅ A. First Non-Repeating Character in a Stream

🧠 Problem Idea

Given a **stream of characters**, after each character arrives, print the **first non-repeating character** seen so far.

Example

Input stream: a a b c

Step-by-step:

Character	Queue	Output
a	a	a
a	(removed)	-1
b	b	b
c	b c	b

Output:

```
a -1 b b
```

🔹 Logic

Use two structures:

- **Queue** → maintain order of characters
- **Frequency array/map** → track how many times each character appears

Steps:

1. Increase frequency of the character
2. Insert character into queue
3. Remove characters from front if they are repeated
4. Print front of queue

✅ B. Circular Queue (Concept Introduction)

🧠 Problem

In a **normal array queue**, space is wasted after dequeues.

Example:

```
[ ][ ][ ][ ][ ]
 ^         ^
front     rear
```

Even if front moves forward, new elements cannot be inserted when rear reaches the end.

🔹 Solution: Circular Queue

A **Circular Queue** connects the end of the array back to the beginning.

This allows reuse of empty spaces.

Circular Movement Formula

```
rear = (rear + 1) % size
front = (front + 1) % size
```

Example:

```
Array size = 5
rear = (4 + 1) % 5 = 0
```

Now insertion starts again from the beginning.

Benefits

✔ Efficient memory usage
✔ Avoids unnecessary overflow
✔ Useful in scheduling systems

✅ C. Queue in BFS (Preview for Graph Week)

🧠 What is BFS?

Breadth First Search (BFS) is a graph traversal technique.

It explores nodes **level by level**.

Why Queue is Used?

Because BFS needs **FIFO processing**.

The first discovered node must be processed first.

Basic Idea

1. Start from a node
2. Insert it into queue
3. Visit neighbors
4. Add neighbors into queue
5. Repeat until queue becomes empty

Visualization

```
Level 0: A
Level 1: B  C
Level 2: D  E  F
```

BFS processes nodes **level-wise**.

Pseudocode Example

First Non-Repeating Character

```
for each character in stream:
    increase frequency

    enqueue character

    while queue not empty AND frequency(front) > 1:
        dequeue

    if queue empty:
        print -1
    else:
        print front
```

Syntax Snippets

C

Since C does not have a built-in queue, we implement it using an **array**.

```c
#include <stdio.h>
#define SIZE 100

char queue[SIZE];
int front = -1;
int rear = -1;
```

Frequency array:

```c
int freq[26] = {0};
```

C++

```cpp
#include <queue>
queue<char> q;
```

Frequency array:

```cpp
int freq[26] = {0};
```

Java

```java
Queue<Character> q = new LinkedList<>();
```

Frequency array:

```java
int[] freq = new int[26];
```

Python

```python
from collections import deque
q = deque()
```

Frequency dictionary:

```python
freq = {}
```

🔹 Common Mistakes

❌ Accessing front without checking if queue is empty
❌ Forgetting modulo % in circular queue
❌ Removing elements incorrectly from queue
❌ Using stack logic instead of queue logic
❌ Not updating frequency properly in stream problems

🔹 Mini Practice

Try implementing these small exercises:

Implement **Circular Queue using array**

Insert elements and **print queue after K operations**

Simulate a **simple task scheduler** using queue

Example:

```
Task1
Task2
Task3
```

Tasks should execute in **FIFO order**.

✨ **Stay Tuned For Upcoming Contests !!!**